

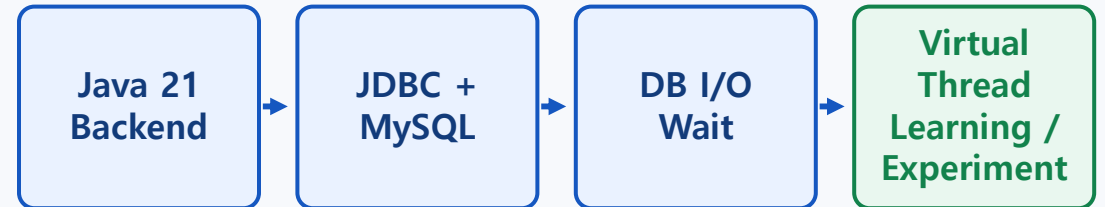
01 Why Java 21 Virtual Threads Matter

Focus on I/O-heavy backend concurrency, not deep performance tuning

Java 21 is the technical focus of this project.

In a JDBC/MySQL backend, DB reads and writes can spend time waiting for responses.

Virtual threads were used to explore how waiting-heavy work can be handled more lightly.



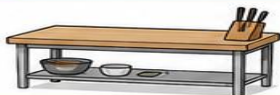
Goal Implement enough to explain data integration and basic concurrency handling in a Java 21 backend.

02 Understanding Stack and Heap with a Kitchen Analogy

Mapping core virtual-thread terms to kitchen roles

Java 21 가상 스레드 주방 비유 그림표

Java 개념을 주방 역할에 대응해서 쉽게 이해하기

Java 개념	주방 비유
Stack	 = →  작업대
Heap	 = →  음식창고
Platform Thread	 = →  작업대를 계속 차지하는 요리사
Virtual Thread	 = →  주문표
JVM Scheduler	 = →  주문을 배정하는 관리자
Carrier Thread	 = →  실제 작업대를 사용하는 실행 스레드
DB I/O 대기	 = →  재료 도착을 기다리는 시간

핵심 이해

- Stack은 지금 일하는 작업 공간입니다.
- Heap은 대기 중인 상태나 객체를 보관하는 공간입니다.
- 가상 스레드는 많은 주문을 효율적으로 처리하는 데 적합합니다.

Note: Stack/Heap are JVM memory concepts, not a Java 17 vs Java 21 difference.

03 Platform Threads vs Virtual Threads

How execution resources behave while waiting for DB I/O



저는 가상 스레드를 성능 튜닝 기술로 깊게 다루기보다는,
Java 21 백엔드에서 I/O 중심 작업을 어떻게 더 가볍게 처리할 수 있는지 이해하는 데 초점을 두었습니다.

플랫폼 스레드 (Platform Thread) 모델



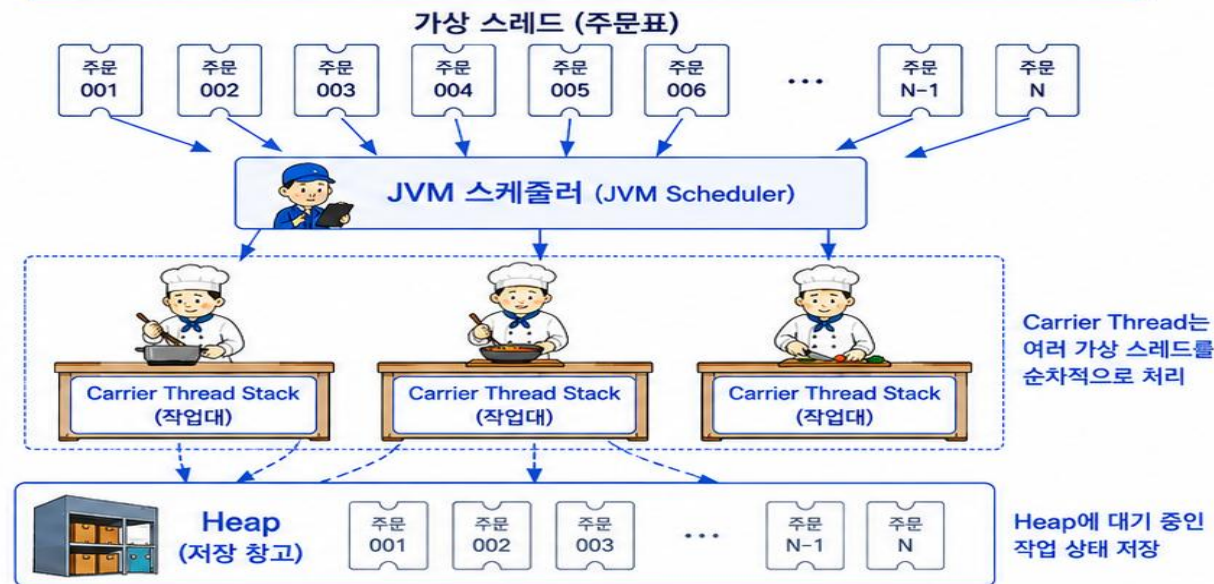
- Stack은 지금 요리하는 작업대이고 Heap은 객체를 보관하는 음식 창고입니다.
- 기존 플랫폼 스레드 방식에서는 요리사마다 작업대를 하나씩 차지합니다.
- DB 응답을 기다리는 상황이면 실제 요리를 하지 않아도 작업대는 계속 묶여 있습니다.



JDBC / MySQL
컨텍스트

- 본 프로젝트는 JDBC와 MySQL을 사용합니다.
- DB 조회나 저장은 CPU 계산보다 DB 응답을 기다리는 시간이 발생할 수 있습니다.
- 이런 I/O 대기 작업에서는 스레드가 계속 OS 스레드를 붙잡고 있으면 비효율적일 수 있습니다.
- Java 21 가상 스레드는 대기 중인 작업의 실행 상태를 관리하고, Carrier Thread를 다른 작업에 사용할 수 있게 하므로 I/O 중심 백엔드 구조를 이해하는 데 의미가 있습니다.

Java 21 가상 스레드 (Virtual Thread) 모델



- Java 21의 가상 스레드 방식에서는 주문표에 해당하는 가상 스레드를 많이 만들 수 있습니다.
- 작업이 DB 응답을 기다리게 되면, JVM이 그 실행 상태를 관리합니다.
- 작업대에 해당하는 Carrier Thread는 다른 가상 스레드를 처리할 수 있습니다.
- 그래서 DB I/O나 외부 API 호출처럼 기다리는 시간이 많은 백엔드 작업에서 가상 스레드가 의미 있습니다.



I/O 중심
백엔드 작업에
적합

Key point: Virtual threads do not speed up the DB; they handle I/O-heavy concurrent work more lightly.

04 Project Application and Experiment Goal

Observing concurrent flow in a Java 21 + JDBC + MySQL experiment

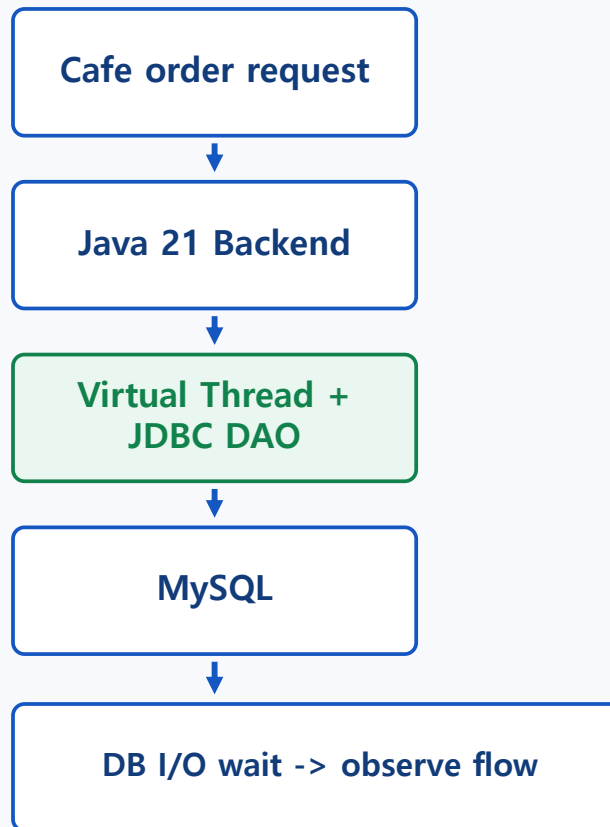
Project Meaning

DB reads and writes include I/O wait for DB responses, not just CPU work.

So this project assumes multiple DB tasks and observes the virtual-thread flow.

Experiment Goal

- 1 Create multiple order tasks
- 2 Run each task on a virtual thread
- 3 Reproduce DB I/O wait
- 4 Check start/end flow in console logs



What to Check

- Thread name
- Task start time
- DB wait section
- Task finish time

Goal: understand I/O-heavy concurrent processing, not prove performance gains

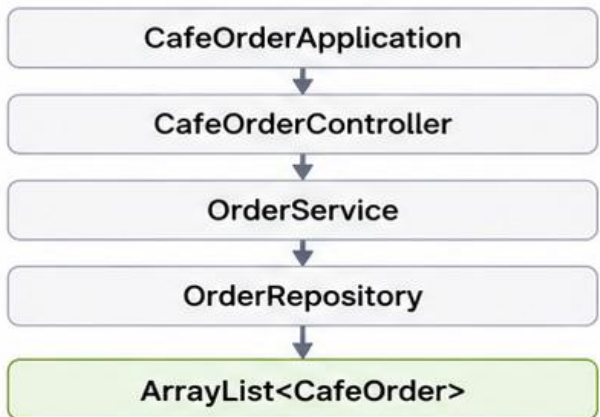


Java 21

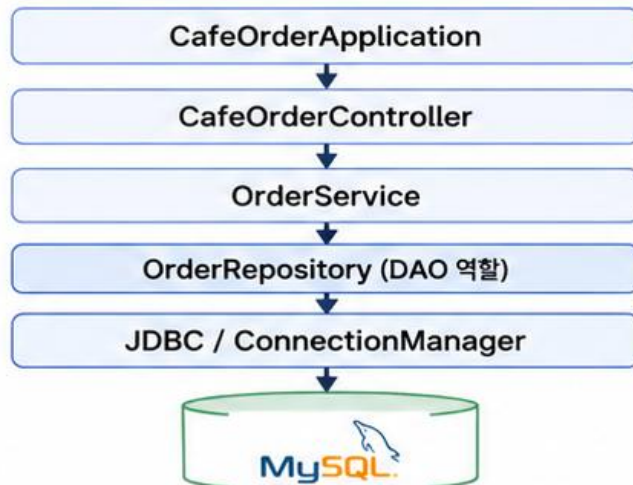
기존 CRUD의 JDBC/MySQL 전환 흐름

기존 CRUD 흐름은 유지하고, 데이터 저장 위치만 Java 메모리 ArrayList에서 MySQL 데이터베이스로 변경했다.

Before (기존 구조)



After (변경된 구조)



항목	기존 방식	변경 방식
저장 위치	Java 메모리 ArrayList	MySQL 테이블
저장 담당	OrderRepository	OrderRepository가 DAO 역할 수행
주문 등록	orders.add(order)	INSERT
주문 조회	Stream/filter	SELECT, JOIN, WHERE
주문 수정	객체 값 변경	UPDATE
주문 삭제	ArrayList에서 제거	DELETE
데이터 유지	프로그램 종료 시 사라짐	DB에 저장되어 유지
자원 관리	별도 필요 없음	Connection, PreparedStatement, ResultSet 반환
안정성	단일 메모리 작업	Transaction, rollback 처리



이번 수정의 핵심은 CRUD 기능을 새로 만드는 것이 아니라,
기존 CRUD의 **저장 계층**을 ArrayList에서 JDBC/MySQL 기반으로 전환한 것이다.



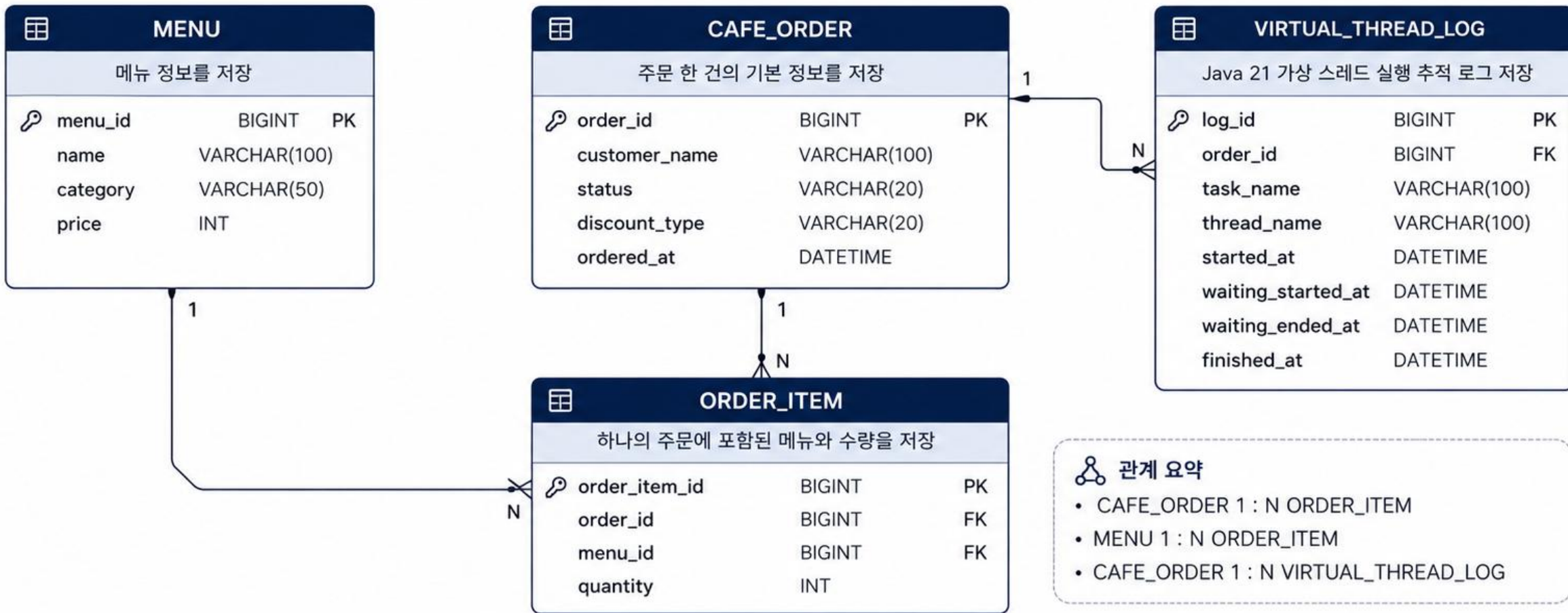
Java 21

ERD 설계 핵심

본 프로젝트의 ERD는 기존 카페 주문 CRUD를 MySQL과 연결하고,
Java 21 가상 스레드의 I/O 대기 흐름을 기록할 수 있도록 설계했다.

설계 목적

- 카페 주문 도메인 CRUD 지원
- Java 21 가상 스레드 실행 로그 저장 (I/O 대기 흐름 관찰)



카페 주문 도메인은 단순히 유지하고, Java 21 가상 스레드 실행 로그를 통해
I/O 중심 백엔드 흐름을 관찰할 수 있도록 설계했다.



Java 21

FK 설계 핵심: ORDER_ITEM

ORDER_ITEM은 주문과 메뉴 사이의 다대다 관계를 풀고, 각 메뉴의 수량을 저장하기 위한 중간 테이블이다.

1. 현실 주문 예시

1번 주문 (고객: 김자바)

아메리카노 2잔,
카페라떼 1잔을 주문했어요!



아메리카노
2잔



카페라떼
1잔



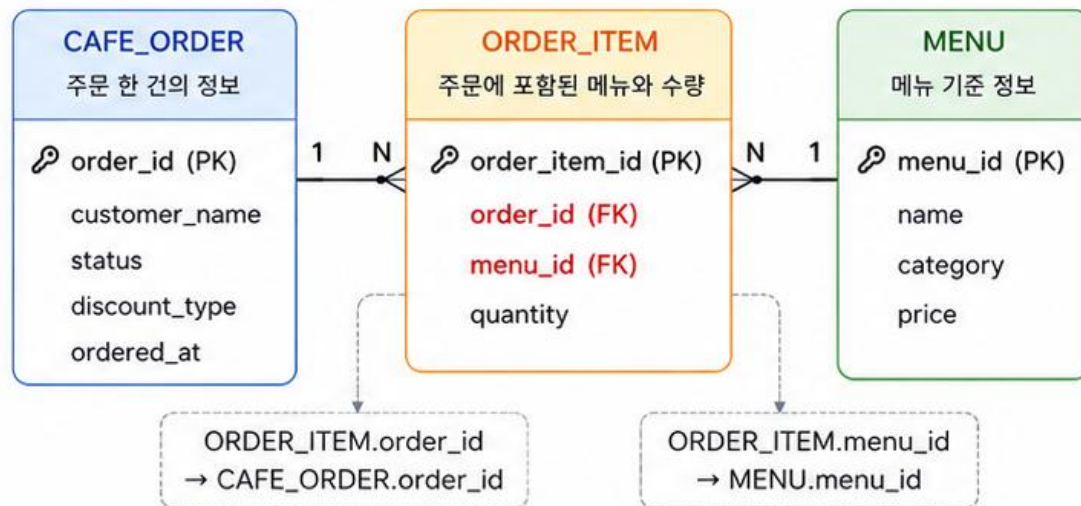
치즈케이크
1개



핵심 포인트

한 주문에는 여러 메뉴가
들어갈 수 있다!

2. 테이블 관계도 (ERD)



예시) 1번 주문의 ORDER_ITEM 데이터

order_item_id (PK)	order_id (FK)	menu_id (FK)	quantity
1	1	1	2
2	1	2	1

1번 주문 → 메뉴 1(아메리카노) 2개, 메뉴 2(카페라떼) 1개

3. FK 설계 설명



CAFE_ORDER

주문 한 건의 기본 정보를 저장한다.
(예: 고객명, 주문 시간, 상태 등)



MENU

판매 가능한 메뉴의 기준 정보를 저장한다.
(예: 메뉴명, 가격 등)



ORDER_ITEM (중간 테이블)

어떤 주문에 어떤 메뉴가 몇 개 포함되었는지를 저장한다.



★ 왜 quantity는 ORDER_ITEM에 저장할까?

quantity는 주문 자체의 속성도 아니고
메뉴 자체의 속성도 아니다.

quantity는 “특정 주문에서 특정 메뉴를
몇 개 선택했는가”라는 관계에서 생기는 속성이다.

그래서 quantity는 ORDER_ITEM에 저장한다.



FK의 역할

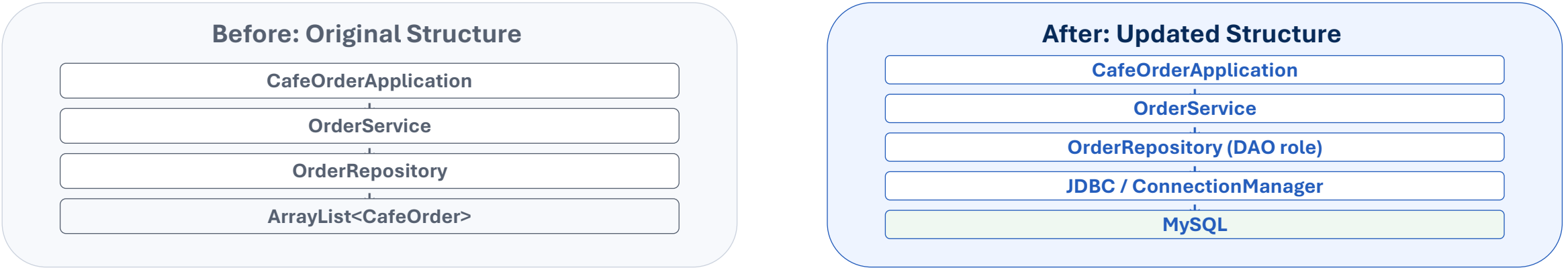
FK는 테이블 사이의 현실 관계를 연결하여
데이터의 일관성과 정확성을 보장한다.



PK가 한 행을 식별한다면, FK는 테이블 사이의 현실 관계를 연결한다.

Storage Change: From ArrayList to MySQL

Only the storage layer changed; the overall application flow stayed the same.



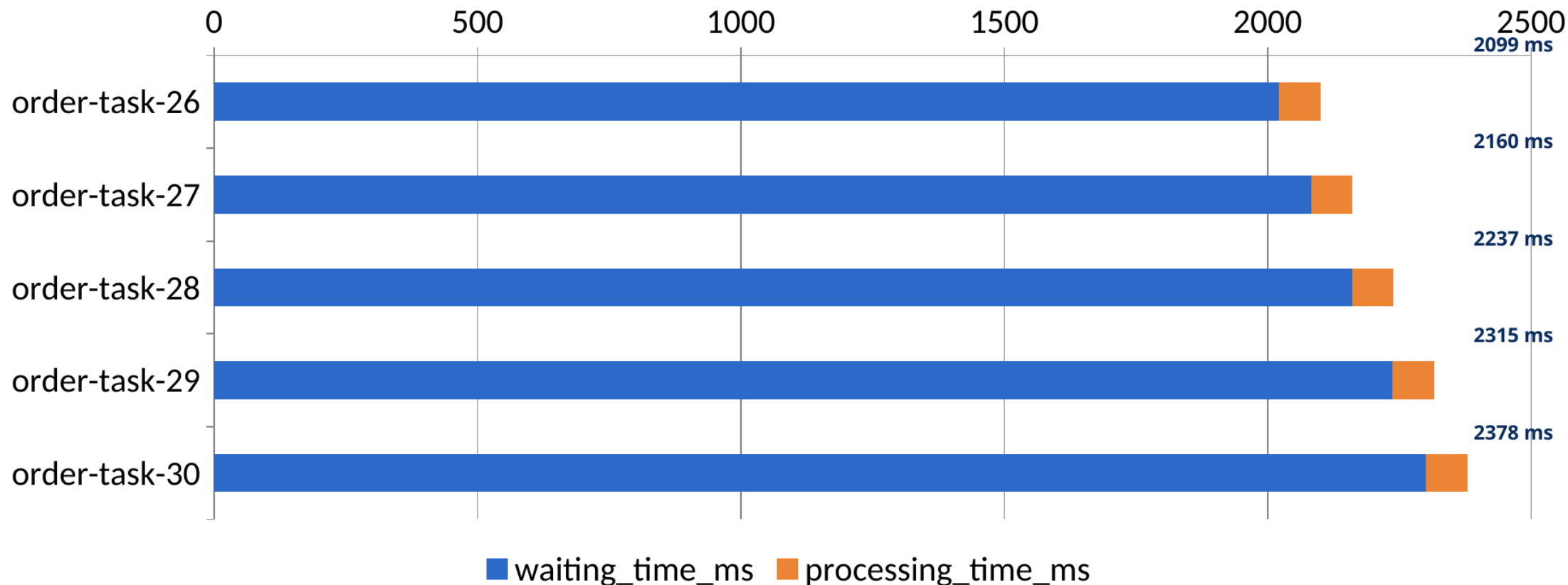
Key: keep the app/service flow and replace only the storage layer.

Item		Before	After
Storage		Stored in ArrayList	Stored in MySQL tables
DB Access		None	Uses JDBC
Storage Owner		OrderRepository	OrderRepository acts as DAO
Create		Memory only	INSERT into cafe_order/order_item
Read		Search in ArrayList	SELECT from MySQL
Update		Change object value	UPDATE MySQL
Delete		Remove from ArrayList	DELETE from MySQL
Persistence		Lost after program exit	Persisted in DB

The full structure was kept; only storage changed from ArrayList to JDBC/MySQL.

실험 결과 : 작업 시간 대부분은 대기 시간

waiting_time_ms + processing_time_ms = total_time_ms



전체 작업 시간 대부분이 waiting_time_ms 로 나타났다 .
즉 , 주문 처리 작업은 CPU 계산보다 DB I/O 대기 비중이 큰 작업임을 확인했다 .

Experiment-Based Scaling Assumption: Waiting Time Can Become a Bottleneck

Experiment Result Average total $\approx 2.2s$ | average waiting $\approx 2.1s$

Most total time was closer to DB I/O wait than CPU work.



Java 17 Platform Thread

10,000 requests / 200 threads = 50 batches

50 processing batches wait in sequence



DB/
API

50 batches x 2.2s

$\approx$ **About 110s**

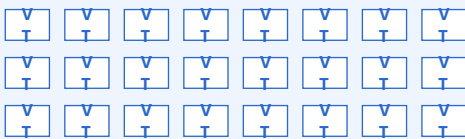
Bottleneck flow

Waiting accumulates by batch -> platform-thread occupancy grows

Experiment
↓
Scale
↓
Interpretation

Java 21 Virtual Thread

Represent 10,000 lightweight request tasks as virtual threads



JVM
Scheduler

Waiting Area



Carrier Threads

C1

C2

C3

C4

Manage waiting work lightly

Reduce OS-thread occupancy pressure

Experiment result: most total_time was waiting_time -> more waiting work increases platform-thread pressure

Interpretation: Java 21 virtual threads can manage I/O-waiting work more lightly.

Note: This is a scaling assumption, not a benchmark. DB throughput, connection count, and network conditions may change real results; Java 21 is not automatically faster.